

Lab 2: Addressing Mode

- **Link**

- [Video](#)
- [HackMD](#)

- **Basic (50%)**

- **Description**

Initialize memory addresses 0x140, 0x141, and 0x142 with three unsigned 8-bit values. Your task is to generate the next three values and store them sequentially at addresses 0x143, 0x144, and 0x145. All memory accesses involved in computing these new values must be performed **using indirect addressing**.

- For each destination address **addr**, determine the new value according to the value currently stored at **addr - 1**. If the value at **addr - 1** is even, compute

$$[\text{addr}] = [\text{addr} - 3] + [\text{addr} - 2] + [\text{addr} - 1].$$

- If the value at **addr - 1** is odd, compute

$$[\text{addr}] = [\text{addr} - 2] - [\text{addr} - 1].$$

-

Note: The values must be generated in increasing address order, from **0x143** to **0x145**. Therefore, a value generated in an earlier step may be used in a subsequent calculation. You may assume that all arithmetic results remain within the unsigned 8-bit range; no overflow or negative result will occur.

- **Example**

Address	Test Case 1	Test Case 2
[0x140]	0x0A	0x05
[0x141]	0x33	0x1E
[0x142]	0x01	0x15
[0x143]	0x32	0x09
[0x144]	0x66	0x0C

[0x145]	0x99	0x2A
---------	------	------

■ **Standard of grading**

1. All read and write operations during the calculation of memory locations [0x143] through [0x145] must use indirect addressing.
2. The generated values must be computed correctly and stored at addresses 0x143, 0x144, and 0x145.

● **Advance (30%)**

■ **Description**

Store two **sorted sequences** A and B in ascending order. Sequence A contains six unsigned 8-bit values stored at addresses [0x200] through [0x205], while Sequence B contains five unsigned 8-bit values stored at addresses [0x210] through [0x214]. Please use **indirect addressing** to access the elements of both sequences. Then, merge sequence A and Sequence B into a single sequence containing all eleven values in descending order. Store the merged sequence sequentially at addresses 0x220 through 0x22A, with the largest value stored at 0x220.

Note: If the current values from Sequence A and Sequence B are equal, the value from Sequence A must appear before the equal value from Sequence B in the merged output. In other words, Sequence A has priority when equal values are encountered. This ensures a **stable merge**.

■ **Example**

Test Case1:

Sequence A: [0x01, 0x03, 0x05, 0x07, 0x09, 0x0B]

Sequence B: [0x02, 0x04, 0x06, 0x08, 0x0A]

Test Case 2:

Sequence A: [0x05, 0x1A, 0x2C, 0x40, 0x7E, 0xA0]

Sequence B: [0x06, 0x20, 0x33, 0x80, 0xF0]

Address	Test Case 1	Test Case 2
[0x220]	0x0B	0xF0
[0x221]	0x0A	0xA0
[0x222]	0x09	0x80
[0x223]	0x08	0x7E
[0x224]	0x07	0x40
[0x225]	0x06	0x33

[0x226]	0x05	0x2C
[0x227]	0x04	0x20
[0x228]	0x03	0x1A
[0x229]	0x02	0x06
[0x22A]	0x01	0x05

■ **Standard of grading**

1. All accesses to the elements of Sequence A and Sequence B must be performed **using indirect addressing**.
2. Sequence A, Sequence B and all computed results must be stored in the **correct and specified memory locations**, with the merging performed using a **stable merge**.

● **Hard (20%)**

■ **Description**

Implement the Quicksort algorithm to sort an unsorted sequence of unsigned 8-bit values in ascending order. The sequence length is not fixed and must be obtained from the specified memory location for each test case.

The sequence length is stored at memory address **0x000**, and the sequence begins at address **0x100**. If the sequence length is **1**, the sequence occupies addresses **0x100** through **0x100 + 1 - 1**. After sorting is complete, the sorted sequence must be stored back in the same memory region beginning at address **0x100**.

Note: The sequence length satisfies $2 \leq 1 \leq 255$, and duplicate values may appear in the sequence.

For convenience when changing test cases, a script for loading test data will be provided on HackMD. You must use the provided script during the demonstration so that test cases can be loaded efficiently and consistently.

■ **Example**

Test Case1:

Sequence length: 0x03

Sequence: [0x03, 0x02, 0x01]

Test Case2:

Sequence length: 0x6

Sequence: [0x0A, 0x34, 0x16, 0x0A, 0xF0, 0xC8]

Address	Test Case 1	Test Case 2
[0x100]	0x01	0x0A

[0x101]	0x02	0x0A
[0x102]	0x03	0x16
[0x103]		0x34
[0x104]		0xC8
[0x105]		0xF0

■ **Standard of grading**

1. All sequence elements must be sorted correctly in ascending order, and the final sorted sequence must be stored in the original memory region beginning at address 0x100.
2. During the demonstration, you must also be able to clearly explain how your implementation works, including the partitioning procedure, index or pointer manipulation, loop control, and the mechanism used to replace recursive Quicksort execution.

■ **Hint**

1. A straightforward recursive implementation of Quicksort may cause a stack overflow because the PIC18F hardware return stack can hold only 31 return addresses. In unfavorable partitioning cases, the recursion depth of Quicksort may approach the sequence length, which can easily exceed this limit. Therefore, you should convert the recursive implementation into an iterative one.
2. Indirect addressing can be used to implement an explicit stack or queue in data memory. In addition to using LFSR to initialize an FSR register, you may directly modify FSRnL and FSRnH when updating an indirect address.
3. You may refer to “Hoare partition scheme” or “Lomuto partition scheme” when designing an in-place Quicksort implementation.